Information Exposure on the Prediction API and in the Extension Console

Divyam Saluja
GitHub: salujadivyam
LinkedIn: salujadivyam

Abstract, The SpoilFree service exposed framework documentation and schema routes by default, did not guarantee uniform responses for unexpected exceptions and logged snippets of classified page text to the browser console during debugging. This report documents the exposure points and the measures taken: disabling documentation routes, returning a generic server error, adding response headers and turning debug logging off by default. Stubbed tests confirmed the expected 404 responses and headers, and console output was checked with debug enabled and disabled. The hosting platform's logs were not inspected, and no exploitation was observed.

Index Terms; *information exposure, security misconfiguration, error handling, logging, FastAPI.*

I. INTRODUCTION

Framework defaults are useful during development, but they can reveal implementation details when a service is deployed without review. Exposure of sensitive information and error messages are recognised weakness classes [1], [2], and security misconfiguration is addressed by the OWASP Top 10 [3]. The information may not be a secret by itself, yet route names, field schemas and internal exception details can make later probing more efficient.

The review identified three separate exposure points in SpoilFree: public API documentation and schema routes, potentially detailed framework error responses and debug logging of page text in the extension console. Each point has a different mitigation and verification method.

II. OBSERVED EXPOSURE

A. Public documentation and schema. FastAPI exposes interactive documentation, an alternative reference page and an OpenAPI schema by default unless these URLs are disabled [4]. The schema can enumerate routes, request fields and response structures. That information helps developers, but it need not be public for the deployed service to perform predictions.

B. Error details. Before the change, the service did not have a global handler that guaranteed a uniform, content-free response for unexpected exceptions. An exception in the prediction path could therefore reach the framework's normal error handling. Detailed errors may disclose internal implementation information and create inconsistent responses [5]. The available review did not establish that a specific secret was returned; the concern was the absence of a guaranteed generic error path.

C. Browser-console logging. During debugging, the content script logged the first 40 characters of each classified text together with the title, confidence and bar value to the console associated with the visited page. Even a short snippet can contain private content, especially on pages where users read personal messages or other non-public text. The issue was the presence of user-derived content in a diagnostic channel, rather than evidence that the console was remotely accessible.

III. IMPLEMENTED MEASURES

A. Disable documentation endpoints. The documentation, reference and schema URLs were set to `None`, preventing the standard routes from being served at their default locations. This reduces the amount of implementation detail available through unauthenticated browsing, but it does not hide other application routes or provide access control.

B. Standardise unexpected errors. A global exception handler now returns a fixed message with status 500. A generic response makes the public error path less informative and more consistent. Internal diagnostics should still be handled carefully in server-side logs, with sensitive request content excluded where possible.

C. Add response headers. Responses carry `X-Content-Type-Options: nosniff` and `Cache-Control: no-store`. The first tells browsers not to infer a different content type, while the second asks caches not to store the response. These headers support safer response handling and reduce the chance that prediction responses remain in a cache; they do not replace validation or authentication.

D. Gate console output. The extension uses a constant named `debug`, which defaults to `false`. Logging is enabled only during testing, and the README identifies turning it off as a release step. This reduces the chance that text snippets and classification details appear during normal use. The release process should treat the default and any temporary debug change as reviewable configuration.

IV. VERIFICATION

With a stubbed model and test client, requests to `/docs` and `/openapi.json` returned HTTP 404, and response headers contained the expected `X-Content-Type-Options` and `Cache-Control` values. On a fake page, console output was observed when debug was enabled and was absent when debug was disabled. These checks cover the reported routes, headers and client-side debug switch in the test setup.

The available evidence does not include inspection of the hosting Space's own logs. It also does not demonstrate that every possible exception is free of detail in every deployment path, or that every release build keeps debug disabled. Those limits should remain explicit in any release record.

V. RESIDUAL RISK AND RECOMMENDED CHECKS

Disabling documentation and returning generic errors reduce information exposure but do not prevent unauthorised use of an API. The service does not have user accounts, so these changes should not be described as access control. The hosting platform's logs and any reverse-proxy logs may still contain request metadata, and they were not reviewed in this investigation.

Before release, check the deployed documentation routes, trigger a controlled server-side exception and confirm that the public response remains generic. Review the platform's logging and retention settings, and inspect the production build to confirm that debug logging is disabled. Avoid logging page text unless it is essential and consented to. These are recommended follow-up checks where the report does not record completed verification.

VI. CONCLUSION

The changes remove low-cost implementation details from the public API surface and prevent normal extension operation from writing text snippets to the page console. The evidence covers disabled documentation routes, response headers and the debug switch under test conditions. It does not cover the hosting platform's logs or provide server-side access control. Logging review and a deployment-level error check remain appropriate before a broader security claim is made.

REFERENCES

[1] MITRE, "CWE-200: Exposure of Sensitive Information to an Unauthorized Actor." Accessed: Oct. 28, 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/200.html

[2] MITRE, "CWE-209: Generation of Error Message Containing Sensitive Information." Accessed: Oct. 28, 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/209.html

[3] OWASP Foundation, "OWASP Top 10:2021." Accessed: Oct. 28, 2026. [Online]. Available: https://owasp.org/Top10/

[4] S. Ramírez, "Metadata and docs URLs," FastAPI documentation. Accessed: Oct. 28, 2026. [Online]. Available: https://fastapi.tiangolo.com/tutorial/metadata/

[5] S. Ramírez, "Handling errors," FastAPI documentation. Accessed: Oct. 28, 2026. [Online]. Available: https://fastapi.tiangolo.com/tutorial/handling-errors/